

REPORT

DISASTER MANAGEMENT SYSTEM

UCS310-Database Management System

B.E. (2nd Year)

Computer engineering



Group members:

Yash Bhardwaj:1024030671

Nikhil Khosla: 1024030672

Shubhi singh:1024030669

Academic year:

2026-27

ABSTRACT

The Disaster Management System is a database-driven application designed to efficiently manage information related to disasters, affected locations, victims, shelters, resources, and responders. The system replaces manual record-keeping with a structured relational database, enabling faster data retrieval, improved coordination, and better decision-making during emergencies. The implementation uses SQL and PL/SQL concepts such as normalization, constraints, joins, triggers, and transactions to ensure data consistency and integrity.

INTRODUCTION

The Disaster Management System (DMS) is designed to address the critical need for coordinated response during emergencies like floods, earthquakes, and fires. While traditional methods rely on manual record-keeping—which is prone to delays and data duplication—this system utilizes a centralized Relational Database Management System (RDBMS) to ensure data integrity and rapid information retrieval.

Natural and man-made disasters generate vast amounts of data regarding affected areas, victim counts, and resource availability. A centralized digital platform is essential to prevent the fragmentation of this data, which often leads to poor coordination and slow emergency response.

PROBLEM STATEMENT

Disaster management involves handling critical data on locations, victims, shelters, and resources. Existing systems are manual or fragmented, causing inconsistency and delays. A centralized DBMS is required for efficient, accurate, and timely data management.

The system is built to support two primary user roles:

- **Admin:** Responsible for registering disasters, monitoring all locations, and managing high-level resource allocation.
- **Responder/Staff:** Focused on field-level updates, such as victim assistance, updating shelter occupancy, and tracking local resource distribution.

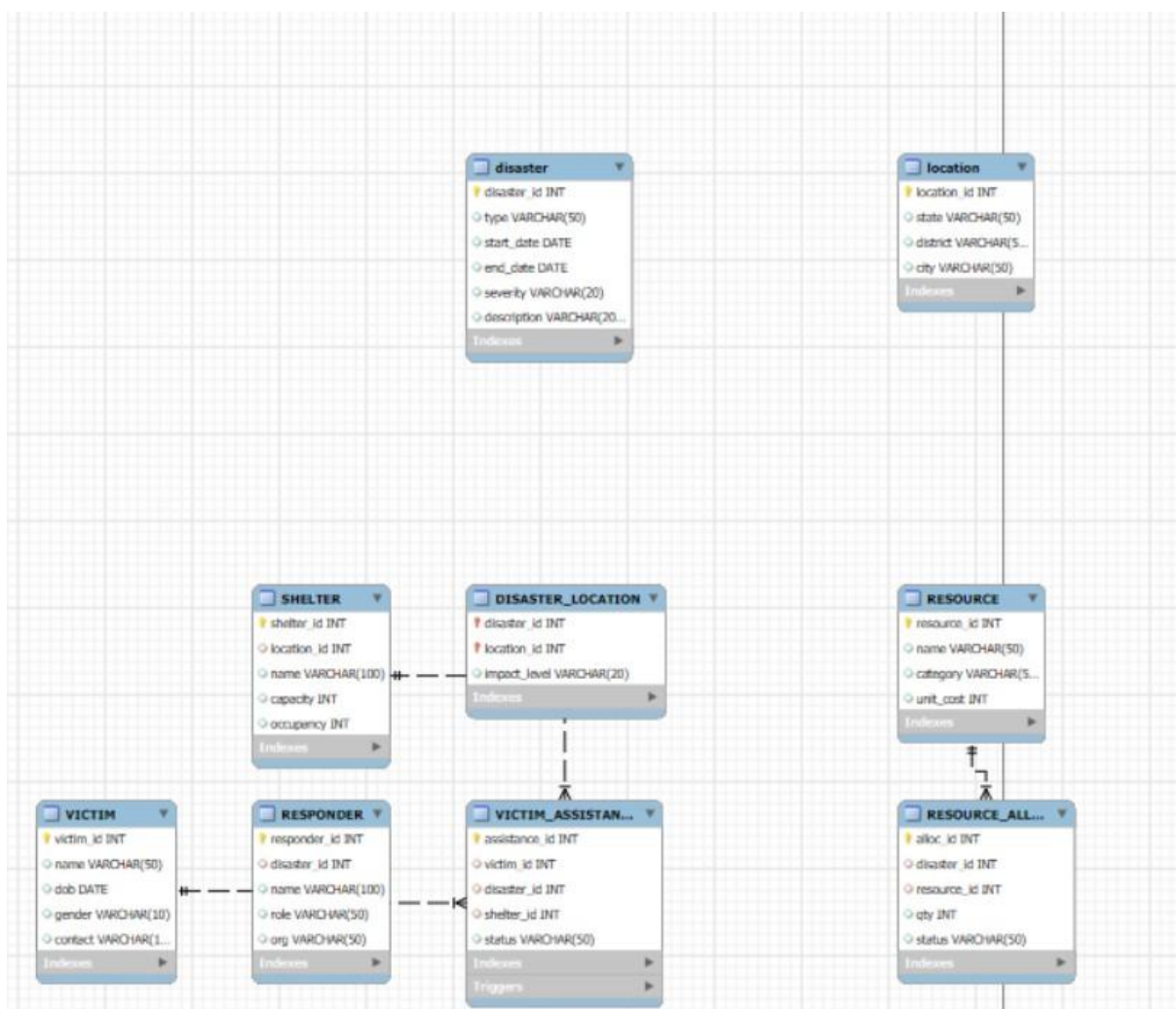
OBJECTIVES

1. To design a well-structured database using Entity–Relationship (ER) modelling to represent entities such as Disaster, Location, Victim, Shelter, Resource, Responder, and their relationships accurately.
2. To convert the ER model into a properly defined relational schema with appropriate Primary Keys, Foreign Keys, and integrity constraints for all 9 tables.
3. To normalize the database schema up to 3NF/BCNF to remove redundancy and maintain logical data dependency.

4. To implement the database using SQL, including DDL statements (CREATE, ALTER, DROP) and DML statements (INSERT, UPDATE, DELETE) for efficient data handling.
5. To develop PL/SQL components such as stored procedures, functions, triggers, and cursors to automate tasks like resource allocation, shelter capacity updates, and disaster record management.
6. To ensure data consistency and reliability using constraints (CHECK, UNIQUE), transaction control statements (COMMIT, ROLLBACK, SAVEPOINT), and proper exception handling.
7. To enable efficient tracking of disaster severity, affected locations, victim assistance, resource distribution, and responder assignments for better decision making during emergencies.

DATABASE DESIGN-

ER DIAGRAM



ENTITIES AND ATTRIBUTES

1. DISASTER

```
• CREATE DATABASE disaster_management;
• USE disaster_management;
• CREATE TABLE disaster (
    disaster_id INT PRIMARY KEY,
    type VARCHAR(50),
    start_date DATE,
    end_date DATE,
    severity VARCHAR(20),
    description VARCHAR(200)
);
```

2. LOCATION

```
11 • CREATE TABLE location (
12     location_id INT PRIMARY KEY,
13     state VARCHAR(50),
14     district VARCHAR(50),
15     city VARCHAR(50),
16     CONSTRAINT unique_location UNIQUE (state, district, city)
17 );
```

3. DISASTER_LOCATION

```
18 • CREATE TABLE DISASTER_LOCATION (
19     disaster_id INT,
20     location_id INT,
21     impact_level VARCHAR(20),
22     PRIMARY KEY (disaster_id, location_id),
23     FOREIGN KEY (disaster_id) REFERENCES DISASTER(disaster_id),
24     FOREIGN KEY (location_id) REFERENCES LOCATION(location_id)
25 );
```

4. VICTIM

```
26 • CREATE TABLE VICTIM (
27     victim_id INT PRIMARY KEY,
28     name VARCHAR(50),
29     dob DATE,
30     gender VARCHAR(10),
31     contact VARCHAR(15)
32 );
```

5. SHELTER

```

33 • CREATE TABLE SHELTER (
34     shelter_id INT PRIMARY KEY,
35     location_id INT,
36     name VARCHAR(100),
37     capacity INT,
38     occupancy INT,
39     FOREIGN KEY (location_id) REFERENCES LOCATION(location_id)
40 );
41

```

6. VICTIM_ASSISTANCE

```

42 • CREATE TABLE VICTIM_ASSISTANCE (
43     assistance_id INT PRIMARY KEY,
44     victim_id INT,
45     disaster_id INT,
46     shelter_id INT,
47     status VARCHAR(50),
48     FOREIGN KEY (victim_id) REFERENCES VICTIM(victim_id),
49     FOREIGN KEY (disaster_id) REFERENCES DISASTER(disaster_id),
50     FOREIGN KEY (shelter_id) REFERENCES SHELTER(shelter_id)
51 );
--

```

7. RESOURCE

```

53 • CREATE TABLE RESOURCE (
54     resource_id INT PRIMARY KEY,
55     name VARCHAR(50),
56     category VARCHAR(50),
57     unit_cost INT
58 );

```

8. RESOURCE_ALLOC

```

60 • CREATE TABLE RESOURCE_ALLOC (
61     alloc_id INT PRIMARY KEY,
62     disaster_id INT,
63     resource_id INT,
64     qty INT,
65     status VARCHAR(50),
66     FOREIGN KEY (disaster_id) REFERENCES DISASTER(disaster_id),
67     FOREIGN KEY (resource_id) REFERENCES RESOURCE(resource_id)
68 );

```

9. RESPONDER

```

70 • CREATE TABLE RESPONDER (
71     responder_id INT PRIMARY KEY,
72     disaster_id INT,
73     name VARCHAR(100),
74     role VARCHAR(50),
75     org VARCHAR(50),
76     FOREIGN KEY (disaster_id) REFERENCES DISASTER(disaster_id)
77 );
--

```

RELATIONSHIPS

1. A Disaster can affect many Locations

Relationship: M : N DISASTER ↔ LOCATION (through DISASTER_LOCATION)

2. A Disaster can have many Victims

Relationship: 1 : N DISASTER → VICTIM_ASSISTANCE

3. A Victim can receive assistance for a Disaster

Relationship: 1 : N VICTIM → VICTIM_ASSISTANCE

4. A Shelter is located in one Location

Relationship: N : 1 SHELTER → LOCATION

5. A Disaster can have many Resources allocated

Relationship: 1 : N DISASTER → RESOURCE_ALLOC

6. A Resource can be allocated to many Disasters

Relationship: 1 : N RESOURCE → RESOURCE_ALLOC

7. A Disaster can have many Responders

Relationship: 1 : N DISASTER → RESPONDER

FOREIGN KEY CONSTRAINTS

- DISASTER_LOCATION.disaster_id → DISASTER.disaster_id
- DISASTER_LOCATION.location_id → LOCATION.location_id
- SHELTER.location_id → LOCATION.location_id
- VICTIM_ASSISTANCE.victim_id → VICTIM.victim_id
- VICTIM_ASSISTANCE.disaster_id → DISASTER.disaster_id
- VICTIM_ASSISTANCE.shelter_id → SHELTER.shelter_id
- RESOURCE_ALLOC.disaster_id → DISASTER.disaster_id
- RESOURCE_ALLOC.resource_id → RESOURCE.resource_id
- RESPONDER.disaster_id → DISASTER.disaster_id

NORMALISATION

Functional Dependencies

- disaster_id → type, start_date, end_date, severity, description
- location_id → state, district, city
- (disaster_id, location_id) → impact_level
- victim_id → name, dob, gender, contact

- shelter_id → location_id, name, capacity, occupancy
- assistance_id → victim_id, disaster_id, shelter_id, status
- resource_id → name, category, unit_cost
- alloc_id → disaster_id, resource_id, qty, status
- responder_id → disaster_id, name, role, org
- 1NF (First Normal Form)
- All attributes are atomic (no multivalued or repeating groups).
- Each table has a defined Primary Key.
- Data is stored in structured tabular form.
- 2NF (Second Normal Form)
- All tables mostly have single-attribute primary keys.
- In the DISASTER_LOCATION table, the composite key (disaster_id, location_id) fully determines impact_level.
- All non-key attributes are fully dependent on their respective primary keys.
- No partial dependencies exist.
- 3NF (Third Normal Form)
- No transitive dependencies are present.
- Disaster, Location, Victim, Shelter, Resource, and Responder details are stored in separate tables to avoid redundancy.

DATABSE IMPLEMENTATION

SQL COMMANDS-

DDL COMMANDS-

CREATE –

```
CREATE TABLE VICTIM_ASSISTANCE (
    assistance_id INT PRIMARY KEY,
    victim_id INT,
    disaster_id INT,
    shelter_id INT,
    status VARCHAR(50),
    FOREIGN KEY (victim_id) REFERENCES VICTIM(victim_id),
    FOREIGN KEY (disaster_id) REFERENCES DISASTER(disaster_id),
    FOREIGN KEY (shelter_id) REFERENCES SHELTER(shelter_id)
);
```

ALTER-

```
ALTER TABLE DISASTER
MODIFY type VARCHAR(30);
```

DML COMMANDS-

INSERT –

```
INSERT INTO disaster VALUES
(1, 'Flood', '2025-07-10', '2025-07-18', 'High', 'Severe flooding due to monsoon'),
(2, 'Earthquake', '2025-03-02', '2025-03-02', 'Medium', 'Moderate tremors'),
(3, 'Cyclone', '2025-10-05', '2025-10-09', 'High', 'Cyclone landfall'),
(4, 'Heatwave', '2025-05-01', '2025-05-20', 'Low', 'Extreme heat');
```

DELETE-

```
DELETE FROM VICTIM
WHERE victim_id = 205;
```

SAMPLE QUERIES-

JOIN

```
-- Resources allocated to each disaster
SELECT d.type, r.name, ra.qty
FROM DISASTER d
JOIN RESOURCE_ALLOC ra ON d.disaster_id = ra.disaster_id
JOIN RESOURCE r ON ra.resource_id = r.resource_id;
```

SUBQUERY

VIEW-

```
SELECT d.type, r.name, ra.qty
FROM DISASTER d
JOIN RESOURCE_ALLOC ra ON d.disaster_id = ra.disaster_id
JOIN RESOURCE r ON ra.resource_id = r.resource_id;
```

[illegible]

PL-SQL COMPONENT –

```
DELIMITER //
```

```
> CREATE PROCEDURE add_disaster (  
    IN p_id INT,  
    IN p_type VARCHAR(50),  
    IN p_start DATE,  
    IN p_end DATE,  
    IN p_severity VARCHAR(20),  
    IN p_desc VARCHAR(200)  
< )  
> BEGIN  
    INSERT INTO disaster  
    VALUES (p_id, p_type, p_start, p_end, p_severity, p_desc);  
  
    SELECT 'Disaster inserted successfully' AS message;  
< END //
```

```
DELIMITER ;
```

```
CREATE TRIGGER update_occupancy  
BEFORE INSERT ON victim_assistance  
FOR EACH ROW  
BEGIN  
    DECLARE cap INT DEFAULT 0;  
    DECLARE occ INT DEFAULT 0;  
  
    -- fetch values safely  
    SELECT capacity, occupancy  
    INTO cap, occ  
    FROM shelter  
    WHERE shelter_id = NEW.shelter_id;  
  
    -- if shelter not found  
    IF cap IS NULL THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Invalid shelter_id';  
    END IF;
```

```
DELIMITER //
```

```
CREATE FUNCTION available_capacity(p_shelter_id INT)  
RETURNS INT  
DETERMINISTIC  
BEGIN  
    DECLARE cap INT;  
    DECLARE occ INT;  
  
    SELECT capacity, occupancy  
    INTO cap, occ  
    FROM shelter  
    WHERE shelter_id = p_shelter_id;  
  
    RETURN cap - occ;  
END //
```

```
DELIMITER ;
```

TRANSACTION MANAGEMENT AND CONCURRENCY-

COMMIT- Use of COMMIT to permanently save changes made to disaster, victim, shelter, and resource records

```
START TRANSACTION;
```

```
INSERT INTO victim VALUES (210, 'Test User', '2000-01-01', 'Male', '9999999999');
```

```
COMMIT;
```

ROLLBACK- Use of ROLLBACK to undo changes in case of errors such as incorrect data entry or failed resource allocation.

```
START TRANSACTION;
```

```
INSERT INTO victim VALUES (210, 'Test User', '2000-01-01', 'Male', '9999999999');
```

```
ROLLBACK;
```

```
SELECT * FROM victim WHERE victim_id = 210;
```

ACID properties-

Atomicity: Each transaction (e.g., victim assistance entry or resource allocation) is completed fully or not executed at all.

```
START TRANSACTION;
INSERT INTO VICTIM_ASSISTANCE
VALUES (500, 201, 1, 301, 'Rescued');
-- If everything OK
COMMIT;
```

Consistency: Database constraints (Primary Key, Foreign Key, CHECK) ensure data remains valid before and after transactions.

```
START TRANSACTION;
```

```
INSERT INTO victim VALUES (210, 'Test User', '2000-01-01', 'Male', '9999999999');
```

Isolation: Multiple users (admin/responders) can work simultaneously without affecting each other's transactions.

```
UPDATE shelter
SET occupancy = occupancy + 1
WHERE shelter_id = NEW.shelter_id;
```

Durability: Once a transaction is committed, changes are permanently stored in the database.

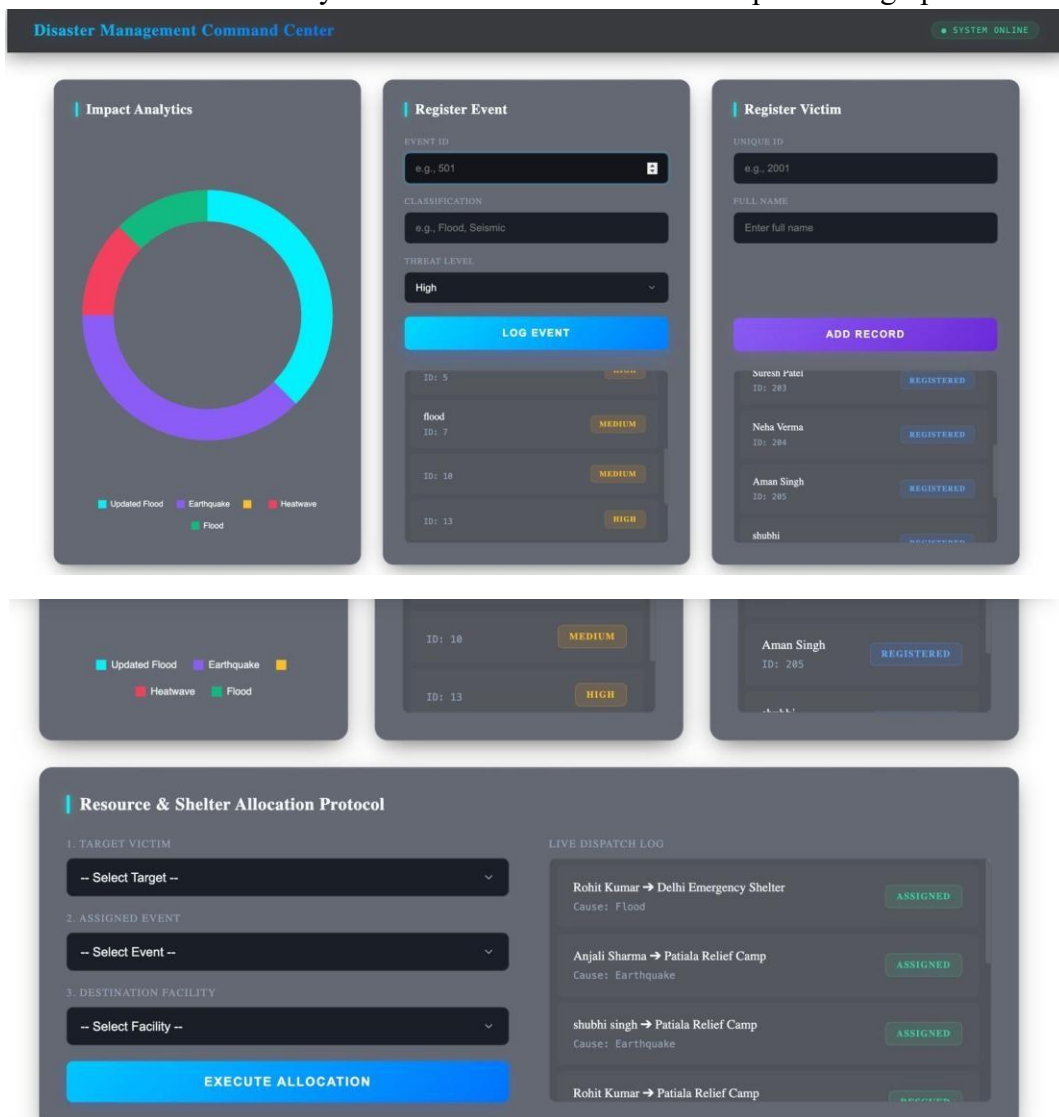
```

START TRANSACTION;
INSERT INTO VICTIM_ASSISTANCE
VALUES (500, 201, 1, 301, 'Rescued');
-- If everything OK
COMMIT;

```

RESULTS AND DISCUSSION-

Frontend interface successfully interacted with the database for performing operations-



The system demonstrates efficient data management by reducing redundancy and ensuring consistency through normalization and constraints. Automated features like triggers improve accuracy and reduce manual effort. The integration of frontend with the database provides a user-friendly interface for managing disaster-related information.

However, the system currently depends on manual data entry and does not support real-time updates or advanced analytics. Despite these limitations, it provides a reliable and scalable foundation for disaster management applications.

Tools & Technologies Used -

MySQL ->Used for designing and managing the relational database for the Disaster Management System.

MySQL Workbench->Used for writing and executing SQL queries, PL/SQL programs, creating tables, managing constraints, and generating reports.

SQL & PL/SQL->Used for database creation (DDL), data manipulation (DML), triggers, procedures, functions, cursors, and transaction management